



UNIVERSITI TEKNOLOGI MALAYSIA

SECP2753

DATA MINING

PROJECT 1

**Title: Student Performance in Secondary Education
(High School) Using Supervised Learning**

PROF. MADYA DR. ROLIANA BINTI IBRAHIM

NO	NAME	MATRIC NO
1.	CHAU YING JIA	A23CS0213
2.	GUI KAH SIN	A23CS0080
3.	SABRINA HENG WEI QI	A23CS0265

Table of Content

1.0 Introduction	2
2.0 Business and Data Understanding.....	4
2.1 Problem Formulation	4
2.2 Dataset Overview	4
3.0 Data Preparation and Preprocessing.....	6
3.1 Overview and Importance of Data Preparation	6
3.2 Data Preparation Steps in Python	7
3.3 Results of Data Preparation	9
4.0 Model Development	9
4.1 Overview of Supervised Learning Methods	9
4.2 Applying the Supervised Learning Algorithms	10
5.0 Model Performance and Evaluation	11
5.1 Evaluation Metrics	11
5.2 Analyze Performance	11
5.3 Confusion Matric Evaluation	13
5.4 Final Evaluation Summary	15
6.0 Conclusion.....	16

1.0 Introduction

Student academic performance is influenced by a variety of factors, including personal background, social environment and school-related conditions. Understanding these factors can help educators and institutions take early action to support students who may be at risk of underachieving.

This project explores student achievement in secondary education using data collected from two Portuguese schools. The dataset includes a wide range of attributes such as student demographics, family background, school support and academic records. The data was gathered through school reports and student questionnaires, covering two subjects: Mathematics and Portuguese.

The goal of this project is to predict students' final grades using supervised machine learning methods. Specifically, we will apply K-Nearest Neighbours (KNN) and Naïve Bayes classification algorithms to determine whether a student is likely to pass or fail. KNN is a

distance-based method that classifies a student based on the outcomes of similar students, while Naive Bayes relies on probability and assumes independence between features.

An important aspect of this dataset is that the final grade (G3) is closely related to the first two period grades (G1 and G2). While using G1 and G2 can improve prediction accuracy, models that exclude them are more practical for early intervention—helping identify at-risk students before they complete most of the school year.

By comparing the performance of classification models on the student performance dataset, the project aims to identify the most effective method for predicting whether a student will pass or fail. The findings are expected to support data-driven decision-making in education, especially for early identification of students at risk of academic failure—allowing timely support and intervention.

2.0 Business and Data Understanding

2.1 Problem Formulation

In education system, accurately identifying students who are likely to perform poorly in their academic assessments is essential for early intervention. Traditional evaluation methods often fail to incorporate the various socio-economic, psychological, and institutional factors that could impact students' performance.

This project aims to develop a predictive model that can determine whether a student will pass or fail based on a range of attributes including demographic details, parental background and others. The two main algorithms used which are the K-Nearest Neighbours and Naïve Bayes are chosen for their simplicity and effectiveness in classification tasks.

2.2 Dataset Overview

The dataset we used is the Student Performance Dataset containing academic, demographic and social attributes of secondary school students. The final grade (G3), originally a numerical score from 0 to 20, has been converted into a binary target (Pass/Fail) to support classification. The dataset includes both categorical and numerical variables. The key details of the dataset are as follows:

Number of Instances: 395 (Math) + 649 (Portuguese) = 1044

Number of Attributes: 33

Target Variable: Final Grade (G3) - (Pass/Fail)

Attribute Summary

Attribute Name	Description	Type
School	student's school ("GP" - Gabriel Pereira or "MS" - Mousinho da Silveira)	categorical
Sex	student's sex ("F" - female or "M" - male)	categorical
Age	student's age (from 15 to 22)	numerical
Address	student's home address type ("U" - urban or "R" - rural)	categorical
famsize	family size ("LE3" - less or equal to 3 or "GT3" - greater than 3)	categorical
Pstatus	parent's cohabitation status ("T" - living together or "A" - apart)	categorical
Medu	mother's education (0 - none, 1 - primary education (4th grade), 2 – 5th to 9th grade, 3 – secondary	numerical

	education or 4 – higher education)	
Fedu	father's education (0 - none, 1 - primary education (4th grade), 2 – 5th to 9th grade, 3 – secondary education or 4 – higher education)	numerical
Mjob	mother's job ("teacher", "health" care related, civil "services" (e.g. administrative or police), "at home" or "other")	categorical
Fjob	father's job ("teacher", "health" care related, civil "services" (e.g. administrative or police), "at home" or "other")	categorical
reason	reason to choose this school (close to "home", school "reputation", "course" preference or "other")	categorical
guardian	student's guardian ("mother", "father" or "other")	categorical
traveltime	home to school travel time (1 - <15 min., 2 - 15 to 30 min., 3 - 30 min. to 1 hour, or 4 - >1 hour)	numerical
studytime	weekly study time (1 - <2 hours, 2 - 2 to 5 hours, 3 - 5 to 10 hours, or 4 - >10 hours)	numerical
failures	number of past class failures (n if $1 \leq n < 3$, else 4)	numerical
schoolsup	extra educational support (yes or no)	categorical
famsup	family educational support (yes or no)	categorical
paid	extra paid classes within the course subject (Math or Portuguese) (yes or no)	categorical
activities	extra-curricular activities (yes or no)	categorical
nursery	attended nursery school (yes or no)	categorical
higher	wants to take higher education (yes or no)	categorical
internet	Internet access at home (yes or no)	categorical
romantic	with a romantic relationship (yes or no)	categorical

famrel	quality of family relationships (from 1 - very bad to 5 - excellent)	numerical
freetime	free time after school (from 1 - very low to 5 - very high)	numerical
goout	going out with friends (from 1 - very low to 5 - very high)	numerical
Dalc	workday alcohol consumption (from 1 - very low to 5 - very high)	numerical
Walc	weekend alcohol consumption (from 1 - very low to 5 - very high)	numerical
health	current health status (from 1 - very bad to 5 - very good)	numerical
absences	number of school absences (from 0 to 93)	numerical
G1	first period grade (from 0 to 20)	Numerical - categorical
G2	second period grade (from 0 to 20)	Numerical - categorical
G3	final grade (from 0 to 20, output target)	Numerical - categorical

3.0 Data Preparation and Preprocessing

3.1 Overview and Importance of Data Preparation

Data preparation plays a critical role in the machine learning process. Raw data often contains inconsistencies, missing values, or non-numeric values that must be handled before applying any algorithm. In this project, we aim to predict whether a student will pass or fail

based on the features provided. Thus, the dataset must be cleaned, encoded, normalized, and split into training and testing sets to ensure optimal model performance.

3.2 Data Preparation Steps in Python

1. Load the two CSV files

We loaded student-mat.csv and student-por.csv using pandas, as both files share the same structure and contain records of students from different subjects.

```
import pandas as pd

# Load student-mat and student-por
mat = pd.read_csv('student-mat.csv', sep=';')
por = pd.read_csv('student-por.csv', sep=';')

# Check their shapes
print("Math dataset shape:", mat.shape)
print("Portuguese dataset shape:", por.shape)
```

2. Add a subject column

We added a subject column to identify the source of each row.

```
mat['subject'] = 'Math'
por['subject'] = 'Portuguese'
```

3. Combine two datasets

We merged both datasets to increase the number of instances from 395 and 649 to a total of 1044, to make the instance size suitable for data mining process.

```
# Combine using concat
df = pd.concat([mat, por], ignore_index=True)

# Check combined shape
print("Combined dataset shape:", df.shape)
```

4. Handle Missing Values

We checked for missing values.

```
df.isnull().sum()
```

5. Create the binary target variable (Discretization)

We converted the final grade into a binary label for classification.

```
# Convert G3 to Pass/Fail
df['pass_fail'] = df['G3'].apply(lambda x: 'Pass' if x >= 10 else 'Fail')

# Preview the changes
df[['G3', 'pass_fail']].head()
```

6. Remove grade columns (Data Reduction)

To prevent data leakage and simulate early intervention, we removed G1, G2 and G3 from the input features.

```
# Remove grade columns
df = df.drop(columns=['G1', 'G2', 'G3'])
```

7. Encode all categorical columns (Transformation)

We encoded all categorical columns into numeric values.

```
from sklearn.preprocessing import LabelEncoder

# Create encoders for each object column
label_encoders = {}

for col in df.select_dtypes(include='object').columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Check that all data is now numeric
df.dtypes
```

8. Split into features (X) and target (Y)

```
# Define features and target
X = df.drop(columns=['pass_fail'])
y = df['pass_fail']
```

9. Normalize features for KNN

```

from sklearn.preprocessing import StandardScaler

# Standardize the feature values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

```

10. Split into training and testing sets

The dataset was split into 70% for training and 30% for testing to evaluate model performance.

```

from sklearn.model_selection import train_test_split

# Split: 70% train, 30% test
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42)

print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)

```

3.3 Results of Data Preparation

After completing the data preparation process, the dataset was successfully transformed and made suitable for supervised learning. The final dataset consists of 1044 student records, combining data from both Mathematics and Portuguese classes. No missing values found, so no imputation or removal was required. The original final grade column (G3) was converted into binary target variable (pass & fail) to support classification tasks.

To avoid data leakage, G1, G2, G3 are removed from the input features. All categorical features were encoded into numeric format using label encoding and numerical features were normalized using standard scaling. Finally, the data was split into training and testing sets with 70%(training) and 30% (evaluation). The resulting dataset is clean, fully numeric and ready for the next step.

4.0 Model Development

4.1 Overview of Supervised Learning Methods

Supervised learning is a machine learning approach where the model learns from labelled data to make predictions. In this project, the target variable is binary, either “Pass” or “Fail” making this a classification problem. We selected two well-known classification algorithms: K-Nearest Neighbors (KNN) and Naive Bayes, due to their simplicity, effectiveness and suitability for small to medium-sized datasets.

- K-Nearest Neighbors (KNN): This algorithm classifies a student based on the outcomes of similar students. It compares the input features like study time, absences, support and finds the nearest students in the dataset to decide the predicted class.
- Naive Bayes: This is a probability-based classifier that uses statistical patterns from the training data to predict the class. It works well even when features are assumed to be independent and is effective for fast and simple classification.

By applying both KNN and Naive Bayes, we are able to compare a distance-based learner with a probability-based learner. This comparison allows us to better understand which type of model is more effective in predicting academic success based on student attributes.

4.2 Applying the Supervised Learning Algorithms

K-Nearest Neighbors (KNN)

1. Import the Model

```
from sklearn.neighbors import KNeighborsClassifier
```

2. Initialize the Classifier. Set $k = 5$

```
knn = KNeighborsClassifier(n_neighbors=5)
```

3. Train the KNN model

```
knn.fit(X_train, y_train)
```

4. Make Prediction

```
y_pred_knn = knn.predict(X_test)
```

Naive Bayes

1. Import the Model

```
from sklearn.naive_bayes import GaussianNB
```

2. Initialize the Classifier. Set $k = 5$

```
nb = GaussianNB()
```

3. Train the KNN model

```
nb.fit(X_train, y_train)
```

4. Make Prediction

```
y_pred_nb = nb.predict(X_test)
```

5.0 Model Performance and Evaluation

5.1 Evaluation Metrics

To evaluate the performance of the classification models which is the K-Nearest Neighbors (KNN) and Naive Bayes, four standard metrics were used:

- Accuracy: The percentage of correctly predicted instances over all predictions.
- Precision: The proportion of positive predictions that were correct.
- Recall: The proportion of actual positive cases correctly identified.
- F1-Score: The harmonic mean of precision and recall, useful for assessing imbalanced data.

These metrics provide a deeper understanding of model quality beyond simple accuracy.

5.2 Analyze Performance

To evaluate the performance of the K-Nearest Neighbors (KNN) and Naive Bayes classifiers, we calculated four core classification metrics: accuracy, precision, recall and F1-score. These metrics help assess the models not just by how often they are correct, but also by how well they balance false positives and false negatives.

Evaluation for KNN

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
# --- KNN Evaluation ---
accuracy_knn = accuracy_score(y_test, y_pred_knn)
precision_knn = precision_score(y_test, y_pred_knn, pos_label=1)
recall_knn = recall_score(y_test, y_pred_knn, pos_label=1)
f1_knn = f1_score(y_test, y_pred_knn, pos_label=1)
confusion_knn = confusion_matrix(y_test, y_pred_knn)
```

Evaluation for Naive Bayes

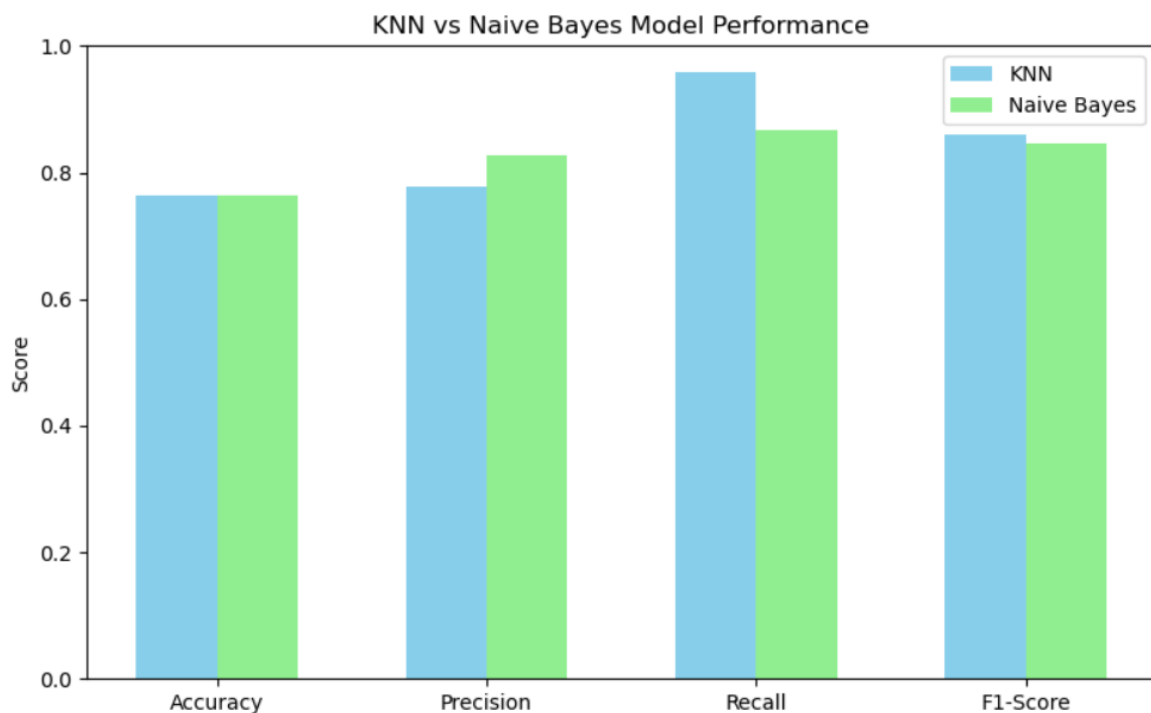
```
# --- Naive Bayes Evaluation ---
accuracy_nb = accuracy_score(y_test, y_pred_nb)
precision_nb = precision_score(y_test, y_pred_nb, pos_label=1)
recall_nb = recall_score(y_test, y_pred_nb, pos_label=1)
f1_nb = f1_score(y_test, y_pred_nb, pos_label=1)
confusion_nb = confusion_matrix(y_test, y_pred_nb)
```

The computed metrics for both classifiers are summarized below:

Metric	KNN	Naive Bayes
Accuracy	0.7643	0.7643
Precision	0.7785	0.8259
Recall	0.9574	0.8681
F1-Score	0.8588	0.8465

The KNN model achieved the accuracy of 76.43%, meaning it correctly predicted performance in over three-quarters of the test cases. It demonstrated strong recall (95.74 %), effectively identifying nearly all students who would pass. However, its precision (77.85%) was slightly lower, suggesting it produced more students predicted to pass who did not. The F1-score (85.88%) shows that the model maintained a good balance between precision and recall.

For Naive Bayes achieved the same accuracy of 76.43 %. However, it has a higher precision (82.59%), meaning it was more cautious in predicting “Pass” and produced fewer false positives than KNN. Its recall and F1-score are lower than KNN.



These results indicate that the KNN classifier slightly outperformed Naive Bayes across all four metrics. In this dataset, KNN demonstrated a stronger ability to identify students who would pass while maintaining a reasonable balance between false positives and false negatives.

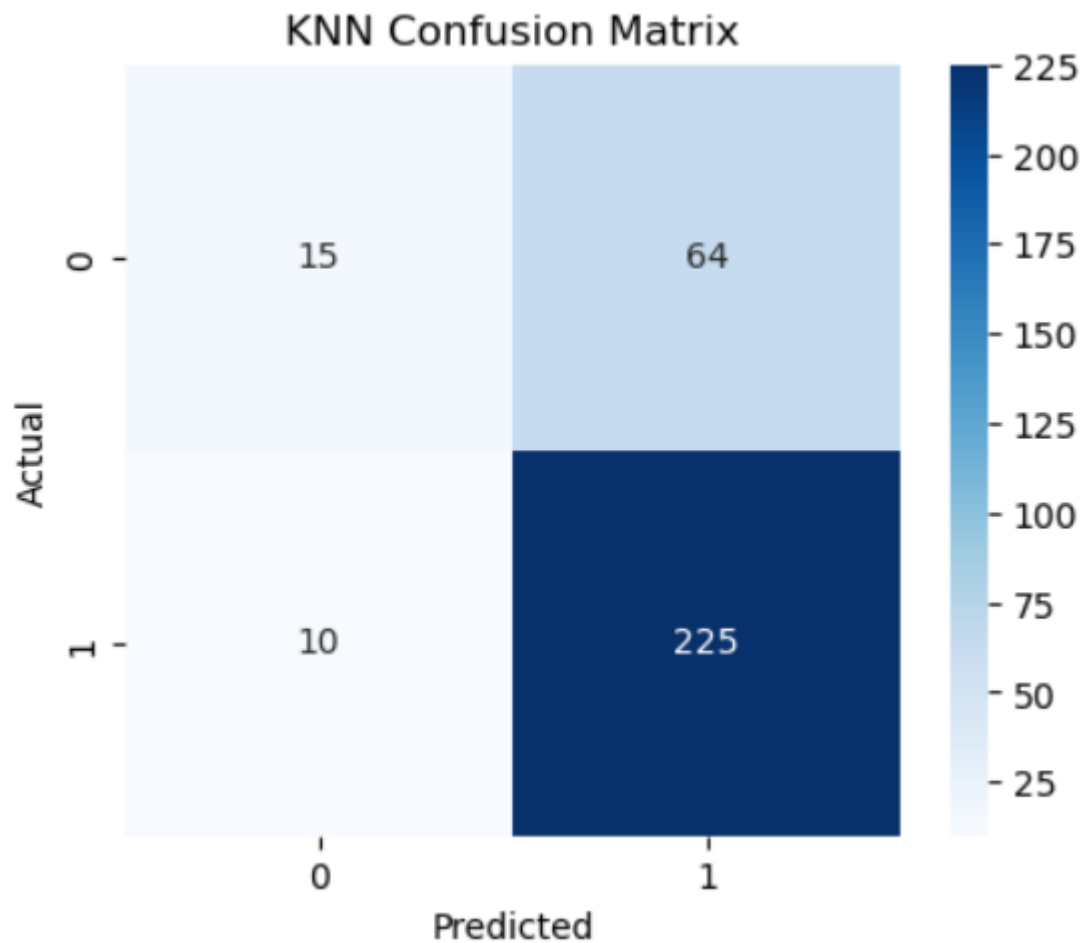
5.3 Confusion Matrix Evaluation

To further analyse the model performance, confusion matrices were plotted for both models. The confusion matrix visualizes the number of correct and incorrect predictions categorized by actual class and predicted class.

Confusion matrix for KNN

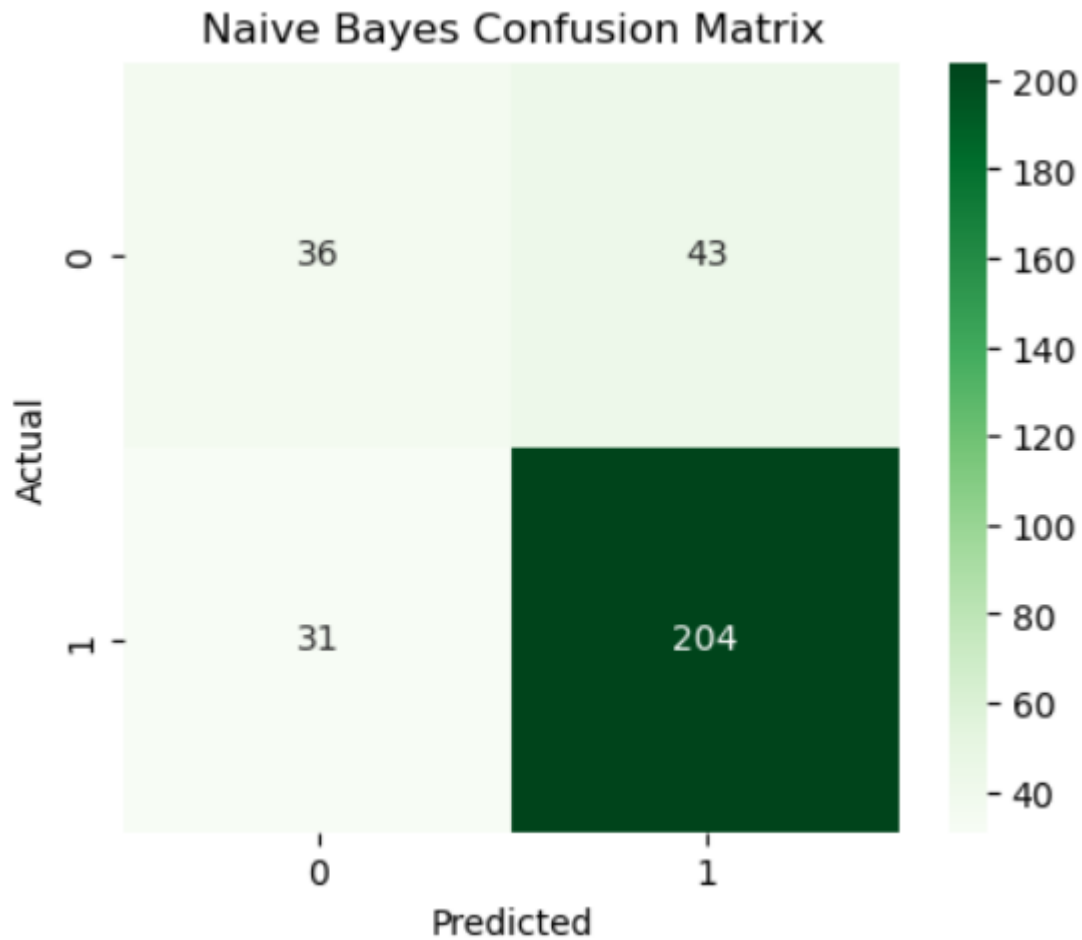
```
import seaborn as sns

# KNN
plt.figure(figsize=(5, 4))
sns.heatmap(confusion_knn, annot=True, fmt='d', cmap='Blues')
plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Confusion matrix for Naive Bayes

```
# Naive Bayes
plt.figure(figsize=(5, 4))
sns.heatmap(confusion_nb, annot=True, fmt='d', cmap='Greens')
plt.title("Naive Bayes Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



In both matrices, the diagonal cells represent correct classifications, while the off-diagonal cells represent misclassifications. KNN’s matrix displayed a slightly higher number of correct “Pass” predictions and fewer misclassifications, reinforcing its slightly superior performance in comparison to Naive Bayes.

5.4 Final Evaluation Summary

In conclusion, both KNN and Naive Bayes classifiers demonstrated the ability to accurately predict student performance using non-grade attributes. However, the KNN model consistently delivered stronger performance across accuracy, precision, recall and F1-score. This suggests that KNN is more suitable for early academic risk prediction, especially in scenarios where feature similarity significantly impacts outcomes.

Nevertheless, Naive Bayes remains a valuable modal due to its computational efficiency and simplicity, making it suitable for real-time or large-scale deployment where speed is prioritized.

6.0 Conclusion

The project aimed to predict student academic performance using supervised machine learning techniques on the Student Performance dataset. The goal was to classify students as either “Pass” or “Fail” based on their background, behaviour and school-related attributes — without relying on final exam grades. The classification task was approached using two algorithms: K-Nearest Neighbors (KNN) and Naive Bayes.

Through thorough data preparation, including merging datasets, label encoding and feature normalization, the dataset was cleaned and transformed into a format suitable for modelling. Both classifiers were trained and evaluated using accuracy, precision, recall and F1-score.

The KNN classifier achieved the highest performance overall, particularly excelling in recall (95.74 %), making it effective at identifying students who were likely to pass. This could be highly valuable in early academic intervention scenarios. Meanwhile, the Naive Bayes classifier offered higher precision (82.59 %), demonstrating a more conservative approach by minimizing false positives.

In summary:

- KNN is better suited for identifying as many successful students as possible, making it ideal for broader educational support programs.
- Naive Bayes, being simpler and faster, is suitable for quick deployment when system efficiency and lower false positive rates are prioritized.

The results highlight the potential of data mining in the education sector to support data-driven decision-making, improve resource allocation and provide timely support to students. With further refinement, such models could be integrated into real-world educational systems to help predict outcomes and inform teaching strategies.